

Kubernetes Network Policy

- 1- Create a Frontend Deployment that runs NGINX image

```
mosalah@dev-boy:~$  
mosalah@dev-boy:~$ kubectl create deployment frontend --image nginx --replicas=1  
deployment.apps/frontend created  
mosalah@dev-boy:~$
```

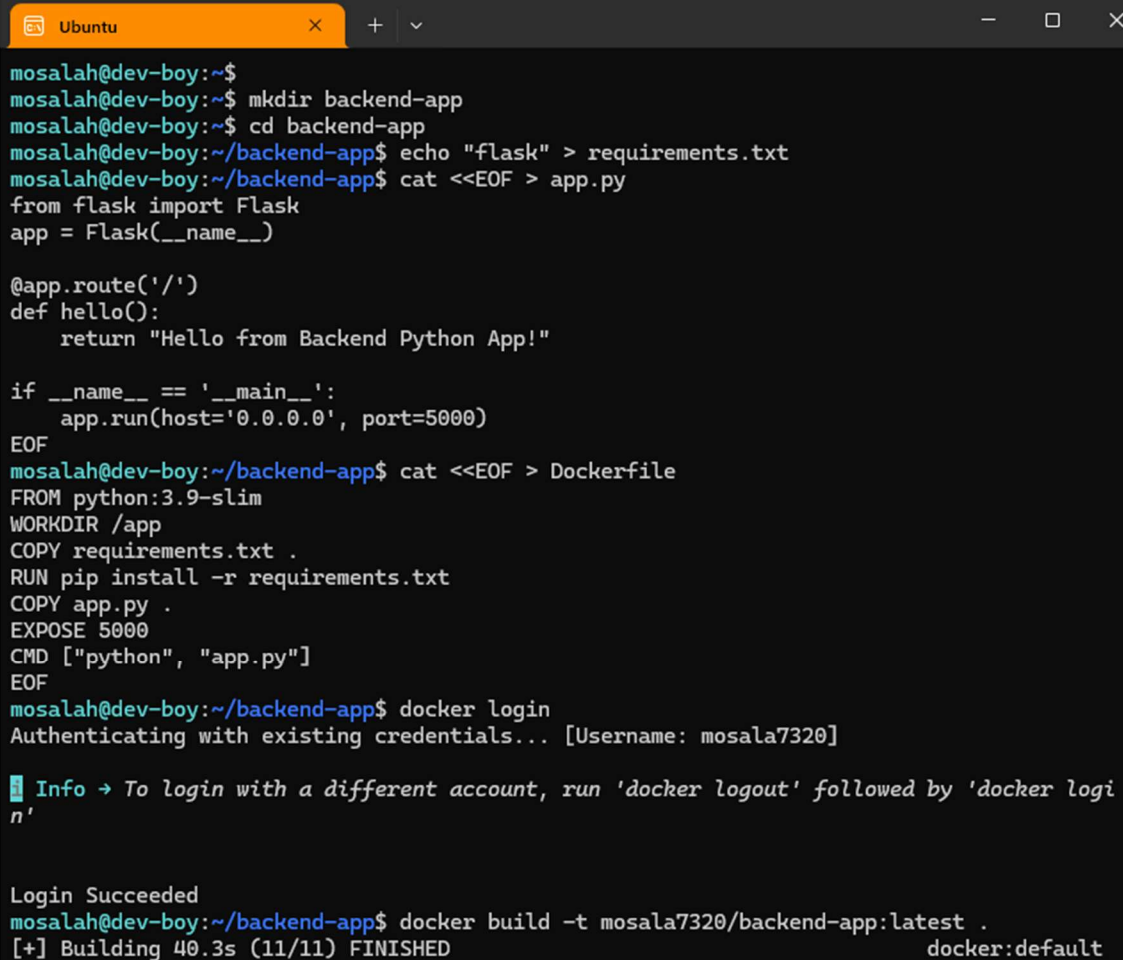
- 2- Create a Frontend NodePort service that exposes your application externally

```
mosalah@dev-boy:~$  
mosalah@dev-boy:~$ kubectl expose deployment frontend --type=NodePort --port=80  
service/frontend exposed  
mosalah@dev-boy:~$
```

- 3- Given the two files:

- app.py
- requirements.txt

build an image using Dockerfile, and push it to Docker Hub



```
mosalah@dev-boy:~$  
mosalah@dev-boy:~$ mkdir backend-app  
mosalah@dev-boy:~$ cd backend-app  
mosalah@dev-boy:~/backend-app$ echo "flask" > requirements.txt  
mosalah@dev-boy:~/backend-app$ cat <<EOF > app.py  
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Backend Python App!"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)  
EOF  
mosalah@dev-boy:~/backend-app$ cat <<EOF > Dockerfile  
FROM python:3.9-slim  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install -r requirements.txt  
COPY app.py .  
EXPOSE 5000  
CMD ["python", "app.py"]  
EOF  
mosalah@dev-boy:~/backend-app$ docker login  
Authenticating with existing credentials... [Username: mosala7320]  
  
Info → To login with a different account, run 'docker logout' followed by 'docker login'  
  
Login Succeeded  
mosalah@dev-boy:~/backend-app$ docker build -t mosala7320/backend-app:latest .  
[+] Building 40.3s (11/11) FINISHED                                docker:default
```

```
Ubuntu x + - □ x

Login Succeeded
mosalah@dev-boy:~/backend-app$ docker build -t mosala7320/backend-app:latest .
[+] Building 40.3s (11/11) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.4s
=> => transferring dockerfile: 184B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.8s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f2 19.4s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f26 0.2s
=> => sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078 251B / 251B 0.2s
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c8 13.88MB / 13.88MB 8.8s
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa0750 1.29MB / 1.29MB 2.0s
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072 29.78MB / 29.78MB 12.8s
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d19 4.0s
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e 0.3s
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710af 1.9s
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c4500783 0.0s
=> [internal] load build context 0.2s
=> => transferring context: 273B 0.0s
=> [2/5] WORKDIR /app 1.2s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install -r requirements.txt 12.3s
=> [5/5] COPY app.py . 0.2s
=> exporting to image 3.0s
=> => exporting layers 1.7s
=> => exporting manifest sha256:fddd4f96e9cb5331ed7a066a54824481767e9e4fee3170a0 0.0s
=> => exporting config sha256:c3b1f741acb43514c67ed380f07a5a0bf5890597cca4f3c962 0.0s
=> => exporting attestation manifest sha256:ff44bf9db5388f38d17134575537a95c1c24 0.1s
=> => exporting manifest list sha256:34ad891f279a8389ea7c0ab631db012f474d243b6de 0.0s
=> => naming to docker.io/mosala7320/backend-app:latest 0.0s
=> => unpacking to docker.io/mosala7320/backend-app:latest 0.9s
mosalah@dev-boy:~/backend-app$

mosalah@dev-boy:~/backend-app$
mosalah@dev-boy:~/backend-app$ docker push mosala7320/backend-app:latest
The push refers to repository [docker.io/mosala7320/backend-app]
a9323cee5c89: Pushed
44136fa355b3: Pushed
b3ec39b36ae8: Pushed
38513bd72563: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
9dd308f1ce7a: Pushed
13d93675239c: Pushed
9f51f3ee0e1b: Pushed
42103aa9051d: Pushed
latest: digest: sha256:34ad891f279a8389ea7c0ab631db012f474d243b6de454ab8783cb78c9698c36
size: 856
mosalah@dev-boy:~/backend-app$
```

- 4- Create a Backend Deployment that uses the pushed image to Docker Hub

```
mosalah@dev-boy:~/backend-app$ kubectl create deployment backend --image=mosala7320/back
end-app:latest --replicas=1
deployment.apps/backend created
mosalah@dev-boy:~/backend-app$
```

- 5- Create a Backend ClusterIP service

```
mosalah@dev-boy:~/backend-app$
mosalah@dev-boy:~/backend-app$ kubectl expose deployment backend --type=ClusterIP --port
=5000
service/backend exposed
mosalah@dev-boy:~/backend-app$
```

- 6- Create a Database Deployment that run mysql image

```
mosalah@dev-boy:~/backend-app$ kubectl create deployment mysql --image=mysql
deployment.apps/mysql created
mosalah@dev-boy:~/backend-app$ kubectl set env deployment/mysql MYSQL_ROOT_PASSWORD=root
deployment.apps/mysql env updated
```

- 7- Create a Database ClusterIP service

```
mosalah@dev-boy:~/backend-app$
mosalah@dev-boy:~/backend-app$ kubectl expose deployment mysql --type=ClusterIP --port=3
306
service/mysql exposed
mosalah@dev-boy:~/backend-app$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
backend   1/1     1            1           21m
frontend  1/1     1            1           44m
mysql     1/1     1            1           5m12s
mosalah@dev-boy:~/backend-app$ kubectl get services
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
backend   ClusterIP   10.106.197.222 <none>        5000/TCP         20m
frontend  NodePort    10.109.241.114 <none>        80:31295/TCP     42m
kubernetes ClusterIP   10.96.0.1      <none>        443/TCP          13h
mysql     ClusterIP   10.100.20.22  <none>        3306/TCP         20s
mosalah@dev-boy:~/backend-app$
```

8- Create a Network Policy that denies all connections

```
mosalah@dev-boy:~/backend-app$ vim network-policy-deny-all.yaml
mosalah@dev-boy:~/backend-app$ vim network-policy-deny-all.yaml
mosalah@dev-boy:~/backend-app$ kubectl apply -f network-policy-deny-all.yaml
networkpolicy.networking.k8s.io/deny-all-traffic created
mosalah@dev-boy:~/backend-app$ |

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-traffic
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress

"network-policy-deny-all.yaml" 9L, 149B
```

- 9- Create an Egress Network Policy that allows the traffic to leave the Frontend Pod

```
mosalah@dev-boy:~/backend-app$ vim network-policy-frontend-egress.yaml
mosalah@dev-boy:~/backend-app$ kubectl apply -f network-policy-frontend-egress.yaml
networkpolicy.networking.k8s.io/frontend-allow-egress configured
mosalah@dev-boy:~/backend-app$
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-allow-egress
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
  - Egress
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: backend
    ports:
    - port: 5000
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    ports:
    - protocol: UDP
      port: 53
```

10- Create an Ingress Network Policy that allows the traffic to enter the Backend Pod

```
mosalah@dev-boy:~/backend-app$ vim network-policy-backend-ingress.yaml
mosalah@dev-boy:~/backend-app$ kubectl apply -f network-policy-backend-ingress.yaml
networkpolicy.networking.k8s.io/backend-allow-ingress created
mosalah@dev-boy:~/backend-app$ |
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-allow-ingress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - port: 5000
```

11- Create an Egress Network Policy that allows the traffic to leave the Backend Pod

```
mosalah@dev-boy:~/backend-app$ vim network-policy-backend-egress.yaml
mosalah@dev-boy:~/backend-app$ kubectl apply -f network-policy-backend-egress.yaml
networkpolicy.networking.k8s.io/backend-allow-egress configured
mosalah@dev-boy:~/backend-app$
```

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-allow-egress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Egress
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: mysql
    ports:
    - port: 3306
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    ports:
    - protocol: UDP
      port: 53
  - to:
    - ipBlock:
        cidr: 0.0.0.0/0

```

"network-policy-backend-egress.yaml" 27L, 480B

12- Create an Ingress Network Policy that allows the traffic to enter the Database Pod

```
mosalah@dev-boy:~/backend-app$ vim network-policy-mysql-ingress.yaml
mosalah@dev-boy:~/backend-app$ kubectl apply -f network-policy-mysql-ingress.yaml
networkpolicy.networking.k8s.io/mysql-allow-ingress configured
mosalah@dev-boy:~/backend-app$
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mysql-allow-ingress
spec:
  podSelector:
    matchLabels:
      app: mysql
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: backend
    ports:
    - port: 3306
```

****Testing The Lab****

- 1- Enter into frontend bash
- 2- Sent request traffic from frontend to backend using (nc) command or (curl) if (nc) doesn't work and as you see backend send message back to frontend
- 3- check if frontend can talk to mysql (database), as you see there is no response, because we denied the traffic from frontend to database

```
mosalah@dev-boy:~/backend-app$
mosalah@dev-boy:~/backend-app$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-5f989cb6b5-n7876            1/1     Running   0           111m
frontend-5dc98f4f6-22fhr            1/1     Running   0           134m
mysql-778ddb6c-htgwx               1/1     Running   0           94m
mosalah@dev-boy:~/backend-app$ kubectl exec -it frontend-5dc98f4f6-22fhr -- bash
root@frontend-5dc98f4f6-22fhr:/# nc -zv backend 5000
bash: nc: command not found
root@frontend-5dc98f4f6-22fhr:/# curl http://backend:5000
Hello from Backend Python App!root@frontend-5dc98f4f6-22fhr:/#
root@frontend-5dc98f4f6-22fhr:/#
root@frontend-5dc98f4f6-22fhr:/# nc -zv mysql 3306
bash: nc: command not found
root@frontend-5dc98f4f6-22fhr:/# curl http://mysql:3306

^Z
[1]+  Stopped                  curl http://mysql:3306
root@frontend-5dc98f4f6-22fhr:/# |

mosalah@dev-boy:~/backend-app$ kubectl get all,networkpolicies
NAME                                READY   STATUS    RESTARTS   AGE
pod/backend-5f989cb6b5-n7876        1/1     Running   0           139m
pod/frontend-5dc98f4f6-22fhr        1/1     Running   0           161m
pod/mysql-778ddb6c-htgwx           1/1     Running   0           122m

NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/backend                     ClusterIP          10.106.197.222  <none>           5000/TCP         137m
service/frontend                     NodePort           10.109.241.114  <none>           80:31295/TCP     159m
service/kubernetes                   ClusterIP          10.96.0.1       <none>           443/TCP          15h
service/mysql                        ClusterIP          10.100.20.22    <none>           3306/TCP         117m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend              1/1     1             1           139m
deployment.apps/frontend             1/1     1             1           161m
deployment.apps/mysql                1/1     1             1           122m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-5f989cb6b5  1         1         1       139m
replicaset.apps/frontend-5dc98f4f6  1         1         1       161m
replicaset.apps/mysql-778ddb6c      1         1         1       122m
replicaset.apps/mysql-7f996486f4    0         0         0       122m

NAME                                POD-SELECTOR        AGE
networkpolicy.networking.k8s.io/backend-allow-egress  app=backend         80m
networkpolicy.networking.k8s.io/backend-allow-ingress app=backend         84m
networkpolicy.networking.k8s.io/deny-all-traffic     <none>              102m
networkpolicy.networking.k8s.io/frontend-allow-egress app=frontend        90m
networkpolicy.networking.k8s.io/mysql-allow-ingress   app=mysql           77m
mosalah@dev-boy:~/backend-app$ |
```